ZYNXIS CLIENT PORTAL

Full-Stack Development Internship

Database: MongoDB Atlas

ODM: Mongoose

Prepared by: Ayeza Waheed

1. Database Overview

The Zynxis Client Portal uses MongoDB Atlas as its database and Mongoose as the Object Data Modeling (ODM) library. The database is organized into five main collections: Users, Clients, Projects, Tasks, and Notifications.

The collections store authentication information, client details, project information, task assignments, and system notifications. Relationships between collections are implemented using MongoDB ObjectId references.

Model	MongoDB Collection	Purpose
User	users	Stores user accounts, authentication information, roles, and profile details
Client	clients	Stores client/company information
Project	projects	Stores projects and their client/user assignments
Task	tasks	Stores tasks associated with projects and users
Notification	notifications	Stores system notifications

Database Technology:

- Database: MongoDB Atlas
- ODM: Mongoose
- Primary Key: MongoDB ObjectId (_id)
- Schema Timestamps: createdAt and updatedAt

Relationships: ObjectId references

2. Collection Schemas

User

Field	Type	Constraints / Description
_id	ObjectId	Primary key / auto-generated
fullName	String	Required, trimmed
email	String	Required, unique, lowercase, trimmed
password	String	Required; stored hashed by application
role	String	Enum: admin, manager, intern; default: "intern"
phone	String	Default: ""
profilePicture	String	Default: ""
isActive	Boolean	Default: true
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

Client

Field	Type	Constraints / Description
_id	ObjectId	Primary key / auto-generated
companyName	String	Required, trimmed
contactPerson	String	Required, trimmed
email	String	Required, lowercase, trimmed
phone	String	Required
address	String	Default: ""
industry	String	Default: ""
status	String	Enum: active, inactive; default: "active"
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

Project

Field	Type	Constraints / Description
_id	ObjectId[]	Primary key / auto-generated
projectName	String	Required, trimmed
description	String	Default: ""
client	ObjectId	Reference to Client; required
assignedUsers	ObjectId[]	Array of references to User
budget	Number	Default: 0
deadline	Date	Optional
status	String	Enum: pending, in progress, completed; default: "pending"
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

Task

Field	Type	Constraints / Description
_id	ObjectId	Primary key / auto-generated
title	String	Required, trimmed
description	String	Default: ""
project	ObjectId	Reference to Project; required
assignedTo	ObjectId	Optional reference to User
priority	String	Enum: low, medium, high; default: "medium"
status	String	Enum: todo, in progress, completed; default: "todo"
dueDate	Date	Optional
attachment	String	Default: ""
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

Notification

Field	Type	Constraints / Description
_id	ObjectId	Primary key / auto-generated
message	String	Required
type	String	Enum: task, project, client; default: "task"
isRead	Boolean	Default: false
createdAt	Date	Automatic timestamp
updatedAt	Date	Automatic timestamp

3. Relationship and cardinality

Relationship	Cardinality	Implementation
Client → Project	One-to-many	Each Project requires one Client; a Client can have zero or many Projects.
Project → Task	One-to-many	Each Task requires one Project; a Project can have zero or many Tasks.
User ↔ Project	Many-to-many	Project.assignedUsers stores an array of User ObjectIds.
User → Task	One-to-Many, optional assignment	A User can have zero or many assigned Tasks; a Task can have zero or one assigned User
Notification	No direct reference	Notification does not contain an ObjectId reference to another collection.

4. MONGOOSE SCHEMA SCRIPT

4.1. User Schema

```
const mongoose = require("mongoose");

const userSchema = new mongoose.Schema(

 {

  fullName: {
```

```
  type: String,
  required: true,
  trim: true,
},
email: {
  type: String,
  required: true,
  unique: true,
  lowercase: true,
  trim: true,
},
password: {
  type: String,
  required: true,
},
role: {
  type: String,
  enum: ["admin", "manager", "intern"],
  default: "intern",
},
phone: {
  type: String,
  default: "",
```

```
    },
    profilePicture: {
      type: String,
      default: "",
    },
    isActive: {
      type: Boolean,
      default: true,
    },
  },
  {
    timestamps: true,
  }
);
module.exports = mongoose.model("User", userSchema);
```

4.2. Client Schema

```
const mongoose = require("mongoose");
const clientSchema = new mongoose.Schema(
  {
    companyName: {
      type: String,
      required: true,
      trim: true,
```

```
},
contactPerson: {
  type: String,
  required: true,
  trim: true,
},
email: {
  type: String,
  required: true,
  lowercase: true,
  trim: true,
},
phone: {
  type: String,
  required: true,
},
address: {
  type: String,
  default: "",
},
industry: {
  type: String,
  default: "",
```

```
    },
    status: {
      type: String,
      enum: ["active", "inactive"],
      default: "active",
    },
  },
  {
    timestamps: true,
  }
);
module.exports = mongoose.model("Client", clientSchema);
```

4.3. Project Schema

```
const mongoose = require("mongoose");
const projectSchema = new mongoose.Schema(
  {
    projectName: {
      type: String,
      required: true,
      trim: true,
    },
    description: {
      type: String,
```

```
  default: "",
},
client: {
  type: mongoose.Schema.Types.ObjectId,
  ref: "Client",
  required: true,
},
assignedUsers: [
  {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
  },
],
budget: {
  type: Number,
  default: 0,
},
deadline: {
  type: Date,
},
status: {
  type: String,
  enum: ["pending", "in progress", "completed"],
```

```
    default: "pending",
   },
  },
  {
   timestamps: true,
  }
);
module.exports = mongoose.model("Project", projectSchema);
```

4.4. Task Schema

```
const mongoose = require("mongoose");

const taskSchema = new mongoose.Schema(
 {
   title: {
    type: String,
    required: true,
    trim: true,
   },

   description: {
    type: String,
    default: "",
   },
```

```
  project: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "Project",
    required: true,
  },
  assignedTo: {
    type: mongoose.Schema.Types.ObjectId,
    ref: "User",
  },
  priority: {
    type: String,
    enum: ["low", "medium", "high"],
    default: "medium",
  },
  status: {
    type: String,
    enum: ["todo", "in progress", "completed"],
    default: "todo",
  },
  dueDate: {
    type: Date,
  },
 attachment: {
```

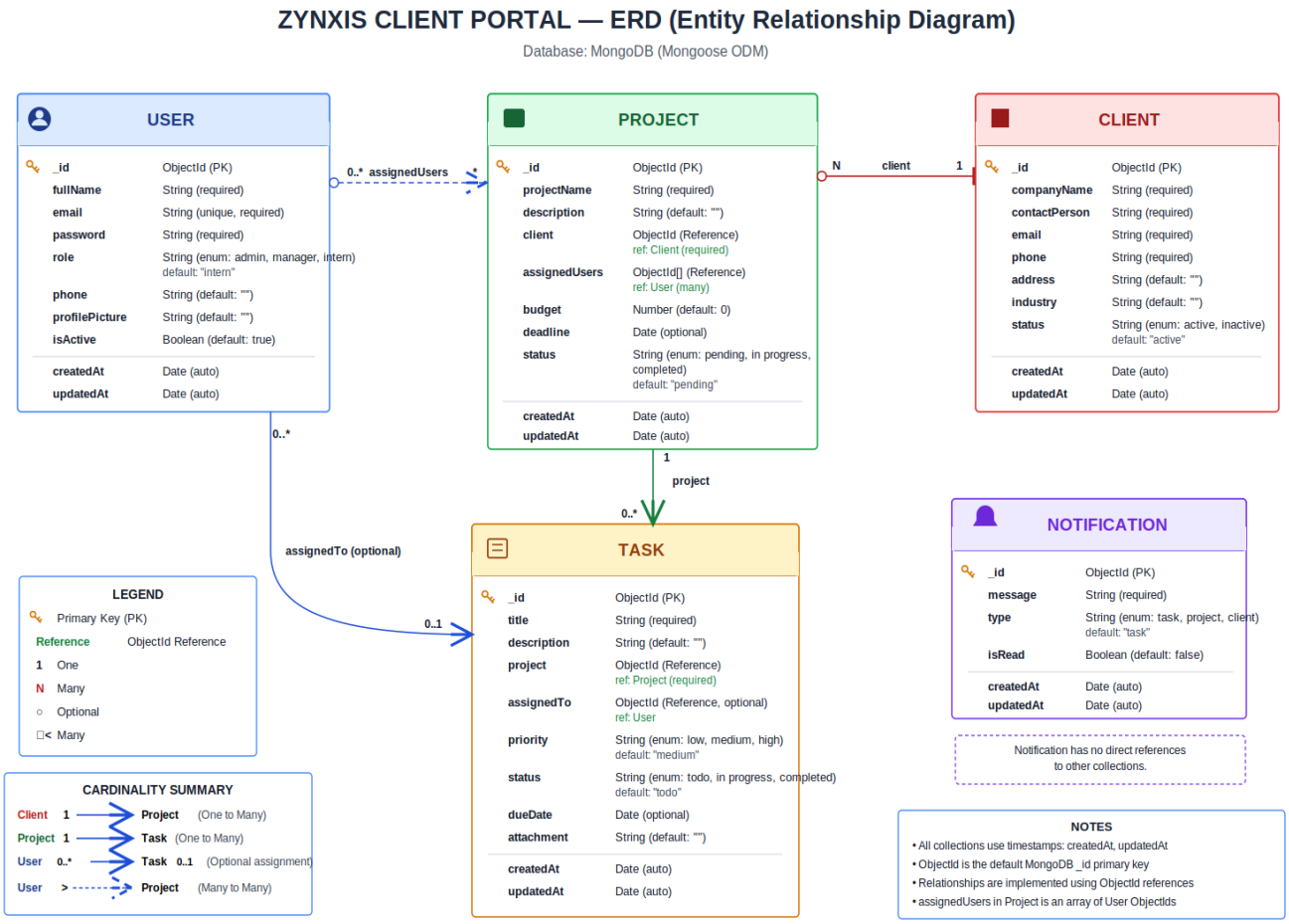
```
    type: String,
    default: "",
   },
  },
  {
   timestamps: true,
  }
);
module.exports = mongoose.model("Task", taskSchema);
```

4.5. Notification Schema

```
const mongoose = require("mongoose");
const notificationSchema = new mongoose.Schema(
  {
   message: {
    type: String,
    required: true,
   },
   type: {
    type: String,
    enum: ["task", "project", "client"],
    default: "task",
   },
  isRead: {
```

```
    type: Boolean,
    default: false,
  },
 },
 {
  timestamps: true,
 }
);
module.exports = mongoose.model("Notification", notificationSchema);
```

5. ENTITY RELATIONSHIP DIAGRAM



The Entity Relationship Diagram represents the database structure of the Zynxis Client Portal. It shows the five MongoDB collections, their fields, ObjectId references, and the cardinality of relationships between the collections.

6. CONCLUSION

The Zynxis Client Portal database has been designed using MongoDB Atlas and Mongoose ODM. The database consists of five main collections: User, Client, Project, Task, and Notification.

The implemented schemas provide validation, default values, timestamps, role information, and ObjectId-based references between related collections. The database structure supports client and project management, task assignment, user roles, and system notifications.

The ERD and schema documentation presented in this document reflect the database structure implemented in the Zynxis Client Portal backend.

6.1. Technology Summary

Database: MongoDB Atlas

ODM: Mongoose

Backend: Node.js + Express.js

Authentication: JWT

Password Hashing: bcrypt

Primary Identifier: MongoDB ObjectId